



Architectural Differences: Vanilla Transformer vs. Modern Large-Language-Model Variants

Below, each subsection first reviews the **vanilla Transformer (2017)** design, then highlights how state-of-the-art LLMs modify or replace that component. Bullet points emphasize the *why* behind each change. Tiny math snippets formalize the idea, and concise PyTorch-style code fragments (simplified) illustrate implementation differences.

1. Post-LayerNorm (a.k.a. "Pre-LN" vs. "Post-LN")

Vanilla Transformer

- Applies LayerNorm **after** the residual addition (Add $\rightarrow$ LN).
- Training a deep stack (>12 layers) is unstable; gradients must flow through two LayerNorms per block.

Modern LLMs

- Swap order to **Pre-Normalize**: LN $\rightarrow$ Sub-Layer $\rightarrow$ Add.
- Gradient scale stays near 1, enabling 48–100+ decoder layers.

Math

Given sub-layer function $F(\cdot)$ and input x :

Vanilla: $y = \text{LN}(x + F(x))$

Pre-LN: $y = x + F(\text{LN}(x))$

Pre-LN keeps the residual branch identity, so $\frac{\partial y}{\partial x} \approx 1$.

```
# Pre-LN block (decoder style)
def block(x):
    x = x + attn(norm1(x))      # self-attention
    x = x + ffn(norm2(x))      # feed-forward
    return x
```

2. Multi-Query Attention (MQA)

Vanilla Transformer

- Each head has its own **Q, K, V**: weight cost $O(hd^2)$.

LLMs

- Single K, V** set shared across heads; only queries remain per-head.
- Memory & compute drop by $\approx 2/3$ on key/value tensors, crucial for long-context decoding.

Math

Let $Q_i \in \mathbb{R}^{L_q \times d_h}$ for head i . In MQA:

$K, V \in \mathbb{R}^{L_k \times d_h}$ (no head index).

Attention for head i : $\text{softmax}(Q_i K^\top / \sqrt{d_h}) V$.

```
# projected once
K = x @ Wk; V = x @ Wv          # share
Q = x @ Wq.reshape(h, d, d_h)  # per head
```

3. Grouped-Query Attention (GQA)

- Compromise between full multi-head and MQA: **g groups** share the same K, V.
- If heads=h and groups=g ($g \ll h$), memory scales as $O(g)$ but keeps representation diversity.

```
groups = 4
K = x @ Wk.reshape(groups, d, d_h)
V = x @ Wv.reshape(groups, d, d_h)
```

4. Key-Value (KV) Cache

Vanilla Transformer

- During autoregressive inference, recomputes all previous attention states each step.

LLMs

- Store past **K, V tensors** once; concatenate only the new token's projection $\rightarrow O(1)$ per-step latency instead of $O(t)$.

```
def decode_step(x_t, cache):
    k_t, v_t = proj_kv(x_t)
    cache['K'] = torch.cat([cache['K'], k_t], dim=1)
```

```
cache['V'] = torch.cat([cache['V'], v_t], dim=1)
out = attn(q=proj_q(x_t), k=cache['K'], v=cache['V'])
```

5. Feed-Forward Layer (FFN) Variants

Vanilla

Two linear layers with ReLU/GELU: $\text{FFN}(x) = W_2 \sigma(W_1 x)$.

Modern Tweaks

- **Gated-GELU/SiLU:**

$$x_{\text{out}} = W_2 (\sigma(W_1 x) \odot W_g x)$$
 — multiplicative gate improves expressiveness.
- **SwiGLU (PaLM):** Uses SiLU as σ .

Code:

```
hidden = F.silu(W1(x)) * Wg(x)  # gate
out = W2(hidden)
```

6. Mixture-of-Experts (MoE)

- Inserts a router that selects **k** experts (mini-FFNs) per token.
- Increases model capacity linearly with #experts while keeping FLOPs roughly constant.

Routing equation

$$y = \sum_{e=1}^E p_e(x) \text{FFN}_e(x),$$

where p_e is top-k softmax gate.

```
scores = router(x)  # [batch, E]
topk = torch.topk(scores, k)
y = (topk.values.unsqueeze(-1) * experts(topk.indices, x)).sum(dim=1)
```

7. RoPE Positional Embedding

Vanilla

- Adds fixed or learned **positional-bias vector** to token embeddings.

RoPE (Rotary Position Embedding)

- Rotates queries & keys in complex plane by angle proportional to position; preserves relative distance in dot-product.

Math

For even indices:

\$\$

$$\begin{pmatrix} q'_{2i} \\ q'_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos \theta_j & -\sin \theta_j \\ \sin \theta_j & \cos \theta_j \end{pmatrix} \begin{pmatrix} q_{2i} \\ q_{2i+1} \end{pmatrix}$$

,

$$\theta_j = \frac{\text{pos}}{10000^{\{2i/d_h\}}}$$

\$\$

Implementation snippet:

```
def apply_rope(q, k, sin, cos):
    q2 = (q * cos) + rotate_half(q) * sin
    k2 = (k * cos) + rotate_half(k) * sin
    return q2, k2
```

8. Multi-Head Latent Attention (MHA-LA)

- Introduces **latent learned queries** separate from token stream (e.g., Perceiver IO).
- Tokens attend to a smaller latent array → quadratic → linear complexity conversion for long context (audio, vision).

```
latent = torch.zeros(L_latent, d).repeat(batch,1,1)
latent = attn(q=latent, k=x, v=x)  # cross attention
```

9. FlashAttention

- An algorithmic improvement, not architectural, but pivotal in LLMs.
- Tiles Q,K,V into SRAM-sized blocks, fuses softmax & matmul, uses causal masking on-chip → bandwidth-optimal $O(n^2)$ with large constant speed-up.

```
# call library kernel
y = flash_attention(Q, K, V, causal=True)
```

Putting It Together: One Modern Decoder Block

```
class LLMBlock(nn.Module):
    def __init__(self, d, h, groups=8):
        super().__init__()
        self.norm = nn.LayerNorm(d)
        self.attn = MultiHeadGQA(d, h, groups, rope=True, flash=True)
        self.ffn = GatedSwiGLU(d, 4*d)
    def forward(self, x, cache=None):
        x = x + self.attn(self.norm(x), cache)
        x = x + self.ffn(self.norm(x))
        return x
```

The block combines **Pre-LN**, **GQA**, **RoPE**, **FlashAttention**, gated FFN, and leverages a **KV cache** during inference. Scaling further swaps the FFN with an **MoE** router.